

Cấu trúc thư mục dự án

Tài liệu này giải thích từng thư mục làm gì, và quan trọng hơn: **cái nào là bắt buộc/quy ước của framework**, cái nào là mình tự đặt ra. Phân biệt được hai loại này giúp biết chỗ nào đổi tên thoải mái, chỗ nào đổi là app chết.

Bản đồ tổng quát

spring-boot-learning/	
├─ .mvn/wrapper/	[Maven Wrapper] pin phiên bản Maven
├─ mvnw, mvnw.cmd	[Maven Wrapper] script chạy Maven không cần cài sẵn
├─ pom.xml	[Maven, BẮT BUỘC] khai báo dependency + build
├─ docker-compose.yml	[tự tạo] PostgreSQL chạy bằng Docker
├─ .gitignore	[Git]
├─ README.md	[tự tạo]
├─ docs/	[TỰ ĐẶT] tài liệu – framework không biết tới
│ └─ request-flow.md	
│ └─ project-structure.md	
├─ target/	[Maven, BẮT BUỘC] output build, đã gitignore
└─ src/	[Maven, BẮT BUỘC]
├─ main/	
├─ java/	[Maven, BẮT BUỘC] source code chính
├─ com/example/crudapi/	[Java, BẮT BUỘC] package ↔ đường dẫn thư mục
├─ CrudApiApplication.java	▲ vị trí có ý nghĩa với Spring
├─ config/	[TỰ ĐẶT]
├─ entity/	[TỰ ĐẶT]
├─ repository/	[TỰ ĐẶT]
├─ service/	[TỰ ĐẶT]
├─ dto/	[TỰ ĐẶT]
├─ web/	[TỰ ĐẶT]
└─ exception/	[TỰ ĐẶT]
├─ resources/	[Maven, BẮT BUỘC] file lên classpath
├─ application.yml	[Spring Boot, BẮT BUỘC về tên + vị trí]
└─ db/migration/	[Flyway, mặc định]
└─ test/	
├─ java/	[Maven, BẮT BUỘC] source code test
├─ com/example/crudapi/	[quy ước mạnh] soi gương package của main

Phần 1 — Những thứ framework quy định (đổi là hỏng)

src/main/java, src/main/resources, src/test/java, target/

Ai quy định: Maven, qua *Standard Directory Layout*.

Maven không cần cấu hình gì vì nó đã mặc định tìm source ở đúng các đường dẫn này. Đây là triết lý **convention over configuration**: cứ đặt file đúng chỗ thì không phải khai báo.

- src/main/java → compile vào target/classes
- src/main/resources → copy nguyên vẹn vào target/classes (tức là lên classpath)
- src/test/java → compile vào target/test-classes, **không** đóng gói vào JAR

Về lý thuyết đổi được bằng <sourceDirectory> trong pom.xml, nhưng thực tế không ai làm — mọi plugin và IDE đều giả định layout chuẩn.

Dự án chưa có src/test/resources. Khi nào cần application-test.yml hoặc file SQL cho test thì tạo thư mục đó — Maven tự nhận, không phải khai báo.

pom.xml

Quy định: Maven. Tên file cố định, phải nằm ở gốc dự án.

Điểm đáng chú ý: khai báo `<parent>` là `spring-boot-starter-parent` nên hầu hết dependency **không cần ghi version** — Spring Boot BOM đã chốt sẵn bộ version tương thích với nhau. Chỉ `springdoc` phải ghi version vì nó là thư viện bên thứ ba ngoài BOM.

mvnw, mvnw.cmd, .mvn/wrapper/

Quy định: Maven Wrapper (không phải Spring).

`maven-wrapper.properties` pin Maven 3.9.16. Người clone về chạy `./mvnw` là có đúng phiên bản Maven đó, không cần cài Maven thủ công, không lo "máy tôi chạy được". `.mvn/` cũng là nơi đặt `jvm.config` hay `maven.config` nếu sau này cần.

com/example/crudapi/ — cấu trúc package

Quy định: Java. Tên package phải khớp đường dẫn thư mục.

`package com.example.crudapi.service;` bắt buộc nằm ở `.../java/com/example/crudapi/service/`. Đây là luật của ngôn ngữ, không phải của Spring.

CrudApiApplication.java — vị trí có ý nghĩa thật sự

Quy định: Spring Boot. Đây là chỗ *dễ sai nhất* với người mới.

Class này nằm ở **package gốc** `com.example.crudapi`, cao hơn tất cả package con. Không phải ngẫu nhiên — `@SpringBootApplication` là gộp của ba annotation:

Annotation gộp bên trong	Hệ quả
<code>@ComponentScan</code>	quét bean từ package chứa class này trở xuống
<code>@EnableAutoConfiguration</code>	bật auto-config theo dependency có trên classpath
<code>@Configuration</code>	bản thân class là một nguồn bean definition

Thêm vào đó, Spring Boot auto-config suy ra hai phạm vi quét khác cũng từ chính package này:

- **Entity scan** (`@EntityScan` ngầm định) — tìm class `@Entity`
- **Repository scan** (`@EnableJpaRepositories` ngầm định) — tìm interface kế thừa `JpaRepository`

→ Nếu lỡ chuyển `CrudApiApplication` xuống `com.example.crudapi.config`, thì `web/`, `service/`, `entity/`, `repository/` đều nằm ngoài tầm quét và app sẽ fail khi khởi động với lỗi kiểu "consider defining a bean of type ...". Quy tắc: class `@SpringBootApplication` luôn ở package cao nhất.

src/main/resources/application.yml

Quy định: Spring Boot. Tên file và vị trí đều là quy ước.

Spring Boot tự tìm file tên `application.properties` hoặc `application.yml` ở 4 vị trí mặc định (classpath root, `classpath:/config/`, thư mục chạy `./`, `./config/`). Ta để ở classpath root — tức là `src/main/resources/`.

File này còn chứa một **profile** `dev` phân tách bằng `---` trong cùng file (bật log SQL). Nếu tách ra file riêng thì phải đặt tên đúng dạng `application-dev.yml` — cũng là quy ước của Boot.

src/main/resources/db/migration/

Ai quy định: Flyway, qua giá trị mặc định `classpath:db/migration`.

Ở dự án này nó còn được khai báo tường minh trong `application.yml` (`spring.flyway.locations: classpath:db/migration`) nên nhìn là thấy ngay, nhưng kể cả xoá dòng đó đi Flyway vẫn tìm đúng chỗ.

Tên file thì bắt buộc tuyệt đối theo cú pháp của Flyway:

```
V1__create_products_table.sql
|  |  |
|  |  | mô tả (dấu _ hiển thị thành khoảng trắng)
|  |  |
|  |  | version, so sánh theo thứ tự số
|  |  |
|  |  | V = versioned migration (còn có R = repeatable, U = undo)
```

Hai dấu gạch dưới giữa version và mô tả là bắt buộc — một dấu là Flyway không nhận ra file.

Lưu ý quan trọng về cách dùng: migration đã chạy rồi thì không được sửa. Flyway lưu checksum trong bảng `flyway_schema_history`; sửa file cũ sẽ khiến lần khởi động sau fail vì lệch checksum. Muốn đổi schema thì thêm `v5__...sql` mới.

src/test/java/com/example/crudapi/ — soi gương package của main

Ai quy định: vừa là quy ước Java, vừa là quy ước Maven Surefire.

Test đặt cùng tên package với class được test (`ProductServiceTest` ở `com.example.crudapi.service`) để truy cập được thành viên *package-private* — thứ mà test ở package khác không thấy.

Hậu tố `Test` trong tên class không phải cho đẹp: Surefire chỉ chạy các file khớp `**/Test*.java`, `**/*Test.java`, `**/*Tests.java`, `**/*TestCase.java`. Đặt tên `ProductServiceSpec` thì `./mvnw test` sẽ lặng lẽ bỏ qua — không báo lỗi, chỉ là không chạy.

Phần 2 — Những thư mục mình tự đặt (Spring không quan tâm)

Đây là phần nhiều người hiểu nhầm nhất: **Spring không nhận diện bean qua thư mục**. Nó quét toàn bộ package con của package gốc rồi đọc **annotation** trên class. Về mặt kỹ thuật, nhét tất cả 30 file vào chung một package `com.example.crudapi` thì app vẫn chạy y hệt.

Các thư mục dưới đây tồn tại vì con người đọc code, không phải vì framework yêu cầu:

Thư mục	Nội dung	Thứ Spring thực sự nhìn vào
web/	ProductController , CategoryController	@RestController
service/	ProductService , CategoryService	@Service + @Transactional
repository/	ProductRepository , CategoryRepository , ProductSpecifications	interface kế thừa JpaRepository
entity/	Product , Category	@Entity
dto/	ProductRequest/Response , CategoryRequest/Response/Summary , PageResponse	<i>không gì cả</i> — record thuần Java
exception/	3 exception + GlobalExceptionHandler	@RestControllerAdvice
config/	JpaAuditingConfig , OpenApiConfig	@Configuration
docs/	tài liệu markdown	<i>không gì cả</i>

Ràng buộc **duy nhất**: chúng phải nằm dưới `com.example.crudapi` để component scan nhìn thấy. Ngoài ra đổi `web/` thành `controller/` , gộp `dto/` vào `web/` — tất cả đều hợp lệ.

Vài lựa chọn đặt tên trong dự án này

- `web/` thay vì `controller/` — cả hai đều phổ biến. `web` là cách Spring tự đặt tên cho module của mình (`org.springframework.web`), và nó bao quát hơn nếu sau này thêm filter, interceptor, hay `@ControllerAdvice` riêng cho tầng HTTP.
- `dto/` chung cho cả request lẫn response — dự án còn nhỏ nên chưa cần tách `dto/request/` và `dto/response/` .
- `ProductSpecifications` nằm trong `repository/` — nó không phải repository, nhưng nó dựng câu truy vấn nên thuộc về mối quan tâm của tầng truy cập dữ liệu. Để ở `service/` cũng không sai, chỉ là kém tự nhiên hơn.
- `exception/` chứa cả exception lẫn handler — `GlobalExceptionHandler` thuộc tầng web xét về chức năng, nhưng gom cùng chỗ với các exception nó xử lý thì dễ theo dõi hơn.

Kiểu phân chia: package-by-layer

Dự án đang chia theo tầng kỹ thuật (layer): tất cả controller một chỗ, tất cả service một chỗ.

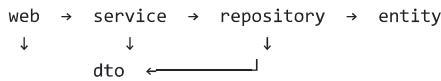
Cách còn lại là **package-by-feature**: chia theo nghiệp vụ.

package-by-layer (dự án này) <pre> ├─ web/ │ ├── ProductController │ └── CategoryController ├─ service/ │ ├── ProductService │ └── CategoryService ├─ repository/ │ ├── ProductRepository │ └── CategoryRepository </pre>	package-by-feature (lựa chọn khác) <pre> ├─ product/ │ ├── ProductController │ ├── ProductService │ ├── ProductRepository │ └── ProductResponse ├─ category/ │ ├── CategoryController │ └── ... </pre>
---	--

Không có cái nào đúng tuyệt đối. Với quy mô hiện tại (2 nghiệp vụ), **package-by-layer** trực quan hơn và khớp với phần lớn tài liệu/tutorial Spring nên dễ học. Khi số nghiệp vụ lên hàng chục, package-by-feature thắng vì mỗi thay đổi chỉ chạm một thư mục thay vì rải rác khắp 5 tầng.

Hệ quả của việc chia tầng: chiều phụ thuộc một chiều

Điều **thật sự** quan trọng không phải là tên thư mục, mà là mũi tên phụ thuộc chỉ đi một chiều:



- web gọi service , **không bao giờ** gọi thẳng repository .
- service không biết gì về HTTP (không `HttpServletRequest` , không `ResponseEntity`).
- entity không rò ra ngoài service — ranh giới ra API luôn là dto .
- repository không chứa business logic.

Không framework nào ép buộc điều này; nó được giữ bằng kỷ luật khi viết code. Vi phạm thì app vẫn chạy, chỉ là mất hết lợi ích của việc chia tầng.

Phần 3 — File hạ tầng ở thư mục gốc

`docker-compose.yml` — tự tạo

Dựng PostgreSQL 18 trong container, map ra cổng **5433** (vì 5432 đã bị Postgres cài sẵn trên máy chiếm). Có healthcheck và named volume `postgres_data` để dữ liệu không mất khi xoá container.

Hoàn toàn độc lập với Spring Boot — app chỉ cần một connection string. Chạy bằng Docker hay Postgres local đều được, chỉ khác biến môi trường `DB_PORT` .

Đáng biết: Spring Boot 3.1+ có module `spring-boot-docker-compose` tự phát hiện file này, tự `docker compose up` và tự cấu hình datasource. Dự án chưa khai báo dependency đó, nên hiện tại việc bật container vẫn là thủ công.

`.gitignore` — Git

Bỏ qua `target/` (output build), thư mục IDE (`.idea/` , `.vscode/` , `.settings/`), log và `.env` . Nguyên tắc: **không commit thứ sinh ra được từ source**, và không commit thứ chỉ đúng với một máy.

`docs/` — tự tạo

Thư mục tài liệu thuần, không framework nào đụng tới. Không nằm trong `src/` nên **không** được đóng gói vào JAR.

Tóm tắt một bảng

Đường dẫn	Ai quy định	Đổi được không?
pom.xml	Maven	✗
src/main/java , src/main/resources , src/test/java	Maven layout	⚠ về lý thuyết được, thực tế đừng
target/	Maven	⚠ cấu hình được, không nên
com/example/crudapi/ khớp tên package	Java	✗
CrudApiApplication ở package gốc	Spring Boot (component scan)	✗ trừ khi tự khai báo <code>@ComponentScan</code> / <code>@EntityScan</code> / <code>@EnableJpaRepositories</code>
application.yml (tên + vị trí)	Spring Boot	⚠ chỉ trong các vị trí Boot hỗ trợ
db/migration/	Flyway (mặc định)	✅ đổi được qua <code>spring.flyway.locations</code>
v1__ten.sql (cú pháp tên file)	Flyway	✗
Hậu tố *Test của class test	Maven Surefire	✅ nhưng phải cấu hình <code><includes></code>
web/ , service/ , repository/ , entity/ , dto/ , exception/ , config/	tự đặt	✅ miễn là còn nằm dưới package gốc
docs/ , docker-compose.yml , README.md	tự đặt	✅

Xem thêm: [request-flow.md](#) — các file này phối hợp với nhau thế nào khi một request đi vào.